

# Pesquisa aprofundada de implementação

Segurança mínima de processo em um runtime estilo Docker

Redução de privilégios antes do `execve()` com `no_new_privs`, `libcap`, `capability bounding set` e `seccomp` básico

Arquitetura-alvo: control plane em Python, fronteira em Cython e data plane em C++

|                              |                                                                                                                                                                                                         |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Objetivo</b>              | Pesquisar e propor a implementação de uma feature de segurança mínima de processo para um runtime estilo Docker, com foco em redução de privilégios antes do <code>execve</code> .                      |
| <b>Contexto arquitetural</b> | Control plane, metadados e API em Python; data plane, comandos e abstrações próximas do kernel em C++; Cython como contrato tipado e camada de integração.                                              |
| <b>Escopo técnico</b>        | <code>no_new_privs</code> , <code>libcap</code> , <code>capability bounding set</code> , redução de privilégios, <code>seccomp</code> básico, desenho arquitetural, DDD, LLD, padrões GoF e requisitos. |
| <b>Data</b>                  | 12 de março de 2026                                                                                                                                                                                     |

Este relatório foi estruturado como pesquisa técnica orientada a implementação. O texto define cada conceito pedido, o contextualiza na pilha Docker/OCI, propõe um desenho para a arquitetura Python + Cython + C++, discute system design, low level design, DDD e padrões GoF, e fecha com requisitos, roadmap e estratégia de validação.

## Síntese executiva

A implementação correta trata `capabilities`, `no_new_privs`, `bounding set` e `seccomp` como um único protocolo de transição de estado. O processo prepara o ambiente, converge `capabilities` com `libcap`, reduz o `bounding set`, marca `PR_SET_NO_NEW_PRIVS`, carrega `seccomp` e só então executa o workload.

## Resumo executivo

A feature pedida não é um detalhe de endurecimento periférico. Em um runtime estilo Docker, ela é a fronteira que separa o código que ainda possui capacidade de configurar o ambiente do código que já deve nascer limitado. O centro da questão é simples de formular e difícil de implementar corretamente: o processo filho precisa executar toda a preparação que depende de privilégios, mas deve chegar ao `execve()` final já incapaz de adquirir privilégios adicionais, já com o capability model estabilizado e já com um filtro seccomp coerente com a política definida pelo control plane. Essa ordem importa porque, em Linux, várias mudanças de privilégio relevantes acontecem justamente no `execve()`, seja por bits `setuid/setgid`, seja por file capabilities. Se o launcher errar essa borda, o sistema cria uma janela em que o workload pode herdar mais poder do que o modelo de segurança pretendia admitir [R1][R2][R3][R9].

No ecossistema Docker, esse problema é tratado como uma responsabilidade da pilha de runtime, e não como uma preocupação espalhada pelo aplicativo do usuário. A API expõe intenções como `--cap-drop`, `--cap-add`, `--security-opt no-new-privileges=true` e um perfil seccomp; a pilha Docker/containerd/runc transforma isso em uma especificação OCI e, por fim, o runtime efetivo materializa capabilities, `noNewPrivileges` e seccomp no processo que fará `execve()`. O desenho proposto para o seu projeto deveria seguir exatamente essa separação de responsabilidades: Python produz intenção, metadados e política; C++ executa a parte irreversível e sensível a kernel; Cython estabiliza o contrato entre ambos [R13][R14][R17][R18][R19].

A conclusão principal desta pesquisa é que a implementação mais segura e sustentável não deve tratar `no_new_privs`, `libcap`, `bounding set` e `seccomp` como quatro knobs independentes. Eles formam um único protocolo de transição de estado do processo. Primeiro vem a preparação privilegiada; depois vem a convergência explícita dos capability sets com `libcap`; em seguida vem a redução irreversível do `bounding set`; logo depois vem `PR_SET_NO_NEW_PRIVS`; por fim entra o filtro seccomp e, somente então, o `execve()` do workload. Qualquer falha em qualquer etapa deve abortar a criação do container de forma fail-closed. A consequência de design é importante: o projeto precisa ter um objeto de domínio imutável, aqui chamado de `LaunchSecurityPlan`, que descreva a política final e também o orçamento temporário de privilégios necessário para chegar até ela.

Também vale uma recomendação de escopo. O primeiro incremento realmente sólido deve privilegiar segurança e previsibilidade, não paridade imediata com todos os casos do Docker. Em termos práticos, isso significa começar aceitando `ambient=[]`, `inheritable=[]`, `no_new_privs=true` por padrão, um conjunto muito pequeno de capabilities finais, e perfis seccomp versionados por arquitetura com `defaultAction=SCMP_ACT_ERRNO`. Casos avançados, como retenção de capabilities após troca de UID ou perfis seccomp altamente especializados por workload, podem entrar em uma segunda fase. Essa abordagem reduz o risco de ambiguidade semântica, preserva a coesão do desenho e produz uma base audível e extensível [R3][R8][R9][R10][R15].

# 1. Escopo, hipótese arquitetural e motivação

Construir um sistema semelhante ao Docker a partir do zero em Python, Cython e C++ exige reconhecer que containerização não é uma única feature, mas um feixe de mecanismos de kernel coordenados por uma arquitetura de software. Namespaces, cgroups, montagem de root filesystem, gerenciamento de imagem, registro de metadados, ciclo de vida de processos e políticas de segurança operam em camadas distintas. A feature desta pesquisa ocupa a camada em que o processo deixa de ser um launcher privilegiado e passa a ser, de fato, o processo inicial do container. É nessa transição que o desenho precisa impedir escalonamento acidental de privilégio e consolidar a política de execução [R1][R3][R9].

Na arquitetura proposta pelo usuário, Python concentra control plane, metadados e API; C++ concentra data plane, comandos e abstrações de baixo nível; Cython faz a cola. Essa divisão é consistente com a história real do Docker. A pilha contemporânea separa a interface e o gerenciamento de estado do runtime efetivo que encosta em syscalls, estruturas de processo e semântica OCI. A lição mais valiosa aqui é que o lado de maior privilégio e menor ergonomia deve ser o menor possível, o mais especializado possível e, idealmente, processualmente isolado do restante do sistema [R16][R17][R19].

Esse ponto é especialmente importante porque a borda de `fork/clone` seguida de preparação e `execve()` não combina bem com um ambiente Python geral, potencialmente multithreaded e rico em estado de runtime. Depois de um `fork()` em um processo complexo, o espaço seguro de operações até o `execve()` é estreito. Por isso, a melhor interpretação do papel do Cython neste projeto não é “permitir que o processo Python faça tudo”; é “fornecer um contrato tipado, estável e verificável para delegar a um runtime C++ o caminho crítico de criação do processo”. Em outras palavras, o seu C++ não deveria ser apenas uma biblioteca de utilidades; ele deveria atuar como um mini-runtime especializado, acionado a partir do control plane, capaz de realizar a sequência privilegiada em um contexto controlado.

A motivação de segurança também precisa ser enquadrada corretamente. `no_new_privs` não substitui capabilities, bounding set nem `seccomp`. Capabilities limitam o que o processo pode fazer; o bounding set limita o que ele ainda pode ganhar em futuros `execve()`; `no_new_privs` bloqueia a obtenção de privilégios adicionais por mecanismos de `execve()`; `seccomp` reduz a superfície de ataque do kernel filtrando syscalls. O valor aparece quando esses elementos são tratados como um pipeline convergente e irreversível, não como opções paralelas [R1][R3][R4][R9].

Por fim, é importante delimitar o escopo negativo. Mesmo uma implementação correta desta feature não produz, sozinha, um sandbox completo. A própria documentação do kernel é explícita ao afirmar que `seccomp` não é um sandbox por si só; trata-se de um mecanismo para reduzir a superfície de ataque. Isso significa que o projeto ainda dependerá, em fases posteriores, de namespaces bem configurados, device policy, talvez user namespaces, e eventualmente AppArmor, SELinux ou Landlock. Ainda assim, a redução mínima de privilégios antes do

`execve()` é uma fundação incontornável, porque ela fecha a fronteira em que o processo deixa de ser um artefato do runtime e passa a ser o workload do usuário [R4][R12][R17].

## 2. Fundamentos conceituais

Em Linux, o `execve()` não é apenas uma troca de imagem de processo. Ele é um ponto de recomputação de credenciais e de aplicação de atributos do arquivo executável. É por isso que os bits `setuid` e `setgid` historicamente alteram a identidade efetiva do processo no momento da execução, e é por isso também que `file capabilities` podem alimentar o conjunto de capacidades permitidas do novo programa. Quando se fala em “reduzir privilégios antes do `execve()`”, o sentido técnico não é apenas “chamar `setuid()` em algum ponto”; é preparar o processo de forma que a chamada final ao executável não consiga reabrir uma janela de privilégio. O desenho correto, portanto, precisa ser pensado a partir da semântica do `execve()`, não apenas da semântica do processo corrente [R1][R2][R9].

O mecanismo `no_new_privs` existe exatamente para endurecer essa borda. Quando um processo marca `PR_SET_NO_NEW_PRIVS` com valor 1, ele assume uma promessa irrevogável ao kernel: chamadas futuras de `execve()` não poderão conceder ao processo privilégios que ele ainda não possuía. Na prática, isso significa que `setuid` e `setgid` deixam de surtir o efeito de elevar identidade, `file capabilities` deixam de ampliar o conjunto permitido, e LSMs não podem afrouxar restrições após o `execve()`. O bit é herdado por `fork`, `clone` e `execve()`, não pode ser revertido e pode ser observado em `/proc/<pid>/status` pelo campo `NoNewPrivs`. Para um runtime de containers, isso transforma uma expectativa de segurança em uma propriedade auditável do processo [R1][R2][R11].

`Capabilities` são a segunda peça do quebra-cabeça. Em vez de tratar “root” como um poder monolítico, o kernel separa privilégios administrativos em bits específicos, como `CAP_CHOWN`, `CAP_SETUID`, `CAP_NET_BIND_SERVICE` ou `CAP_SYS_ADMIN`. A documentação do Docker enfatiza essa granularidade ao explicar que containers não nascem com todos os privilégios do host; eles recebem apenas um conjunto restrito por padrão, podendo sofrer `cap-drop` ou `cap-add` conforme o caso. Esse modelo é mais fino do que o binário `root/non-root` e precisa ser refletido na sua arquitetura, especialmente porque o launcher privilegiado e o workload final não têm necessariamente o mesmo orçamento de `capabilities` [R9][R12][R13].

A semântica das `capabilities`, contudo, não vive em um único conjunto. O kernel trabalha com conjuntos distintos, cada um com papel próprio. O conjunto efetivo é o que de fato vale para checagens de privilégio do processo naquele instante; o permitido funciona como teto a partir do qual o efetivo pode ser habilitado; o herdável participa da transmissão de bits por `execve()` em combinação com atributos do arquivo; o ambiente, mais recente, é carregado através de `execve()` para programas não privilegiados; e o `bounding set` funciona como um teto global para o que ainda pode ser obtido no futuro. Na prática, erros de implementação surgem exatamente quando esses conjuntos são confundidos ou manipulados como se fossem equivalentes [R9][R10][R17][R18].

| Conjunto           | Papel semântico                                                                                                    | Implicação para o projeto                                                                                                 |
|--------------------|--------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>Effective</b>   | Bits consultados pelo kernel nas checagens de privilégio do processo naquele instante.                             | É o conjunto que realmente confere poder operacional. No launch seguro, deve convergir cedo para o orçamento final.       |
| <b>Permitted</b>   | Teto de capabilities que o processo pode ativar no conjunto efetivo.                                               | Deve ser tratado como orçamento potencial. Um runtime previsível evita deixar bits permitidos sem necessidade real.       |
| <b>Inheritable</b> | Conjunto levado em conta na transmissão de capabilities via execve em combinação com atributos do executável.      | No baseline, convém mantê-lo vazio para reduzir propagação implícita entre imagens de processo.                           |
| <b>Ambient</b>     | Capabilities preservadas através de execve para programas não privilegiados.                                       | É a fonte mais fácil de privilégio implícito difícil de explicar. A recomendação para o MVP é <code>`ambient=[]`</code> . |
| <b>Bounding</b>    | Limite superior para capabilities que ainda podem ser ganhas no futuro via execve e teto para adições ao herdável. | É uma cerca irreversível do futuro. Dropar bits aqui é essencial, mas não substitui limpar conjuntos correntes.           |

*Tabela 1 — Semântica dos conjuntos de capabilities e implicações de design*

A biblioteca recomendada para manipular esses conjuntos a partir de C ou C++ é a `libcap`. A própria `capabilities(7)` sugere o uso programático de `cap_get_proc()` e `cap_set_proc()`, e a API da `libcap` expõe explicitamente os flags `CAP_EFFECTIVE`, `CAP_PERMITTED` e `CAP_INHERITABLE`, além de helpers para bounding e ambient capabilities, como `cap_drop_bound()`, `cap_get_ambient()`, `cap_set_ambient()` e `cap_reset_ambient()`. O valor arquitetural da `libcap` não está apenas em poupar chamadas cruas a `capget` e `capset`; ele está em reduzir a chance de inconsistência semântica entre conjuntos, algo especialmente relevante quando o launcher precisa convergir de um estado temporário privilegiado para um estado final estritamente menor [R9][R10].

O capability bounding set merece tratamento isolado porque ele costuma ser mal interpretado. O kernel o define como um limite para as capabilities que podem ser ganhas durante `execve()`, e também como um teto para o que pode ser adicionado ao conjunto herdável via `capset`. Esse conjunto é por thread, é herdado por `fork` e preservado por `execve()`. Mais importante: a remoção é irreversível. `prctl(PR_CAPBSET_DROP)` ou `cap_drop_bound()` podem retirar bits, mas não restaurá-los. Ao mesmo tempo, a documentação deixa claro que remover um bit do bounding set não apaga automaticamente o mesmo bit do conjunto herdável já existente. Em

outras palavras, bounding set não substitui a limpeza explícita dos conjuntos correntes; ele fecha a porta do futuro, mas não reorganiza sozinho o presente [R9][R10].

Esse detalhe produz uma consequência direta no low level design. Para conseguir dropar bounding bits, o processo ainda precisa ter CAP\_SETPCAP efetivo. Logo, a ordem de operações não pode zerar capabilities indiscriminadamente cedo demais. Primeiro, o launcher converge os conjuntos correntes para um perfil temporário suficiente para terminar a preparação; depois, ainda com CAP\_SETPCAP, derruba tudo o que não deve permanecer no bounding set; só então abandona o que ainda restava de poder transitório. Essa é uma das razões pelas quais uma API de alto nível em Python deve expressar a política final, mas o runtime C++ precisa derivar internamente um orçamento temporário de setup.

Seccomp fecha o circuito. Em modo filter, ele instala no kernel um programa BPF que decide, syscall por syscall, o que o processo poderá chamar. A documentação do kernel destaca dois pontos essenciais: o mecanismo reduz a superfície de ataque do kernel, mas não é um sandbox completo; e o filtro é preservado por fork/clone e execve(), desde que permitido pela política. A chamada SECCOMP\_SET\_MODE\_FILTER exige CAP\_SYS\_ADMIN na user namespace do processo ou, alternativamente, que o processo já tenha no\_new\_privs=1; caso contrário, a operação falha com EACCES. É exatamente por isso que no\_new\_privs e seccomp são frequentemente tratados em conjunto e por que o Docker documenta que filtros seccomp mais restritivos podem ser aplicados depois da queda de privilégios [R3][R4][R14].

Para um projeto from scratch, a recomendação prática é usar libseccomp, não BPF cru. A libseccomp permite inicializar um contexto com uma ação padrão, adicionar regras e carregar o filtro de forma mais portátil e auditável. A família seccomp\_init(), seccomp\_rule\_add() e seccomp\_load() cobre a maior parte do que um MVP precisa. A documentação também lembra que múltiplos filtros podem coexistir e que a ação mais restritiva prevalece; isso abre espaço para um design incremental em que um perfil base é sempre aplicado e workloads específicos só conseguem adicionar restrições, nunca relaxá-las [R5][R6][R7][R8].

O conceito de “seccomp básico”, portanto, não deveria ser entendido como uma lista arbitrária e imutável de syscalls, mas como uma política inicial orientada por compatibilidade e risco. Uma boa linha de partida é uma allowlist com SCMP\_ACT\_ERRNO como ação padrão, liberando apenas syscalls ordinárias de inicialização de processo, memória, I/O de arquivos, sinais, relógio e sincronização, e negando explicitamente chamadas conhecidamente perigosas ou incompatíveis com o modelo de container, como bpf, mount, umount2, init\_module, finit\_module, delete\_module, kexec\_load, open\_by\_handle\_at, perf\_event\_open, ptrace, userfaultfd, swapon, swapoff, reboot e, quando não forem necessárias, unshare, setns e variantes de clone que criem novos namespaces. O próprio Docker mantém um perfil default em allowlist e documenta diversas dessas exclusões [R15].

Há, porém, uma nuance que um projeto sério não pode ignorar. O conjunto mínimo real de syscalls varia por arquitetura, por libc e pelo tipo de workload. Um binário dinâmico em glibc pode precisar de uma sequência inicial diferente de um binário estático ou de um processo em musl; além disso, alguns syscalls modernos



aparecem como substitutos de antigos dependendo da versão de kernel. Isso implica que o control plane não deve tratar o perfil seccomp como texto livre ad hoc. Ele deve tratá-lo como artefato versionado por arquitetura, compilado a partir de uma representação de domínio e validado em testes de compatibilidade. Essa decisão transforma uma regra de segurança em um componente de produto, e não em um trecho informal de configuração [R8][R15][R17].

### 3. Contextualização no Docker original e na pilha OCI

O Docker original evoluiu para uma arquitetura em camadas em que a expressão da política e a materialização da política são responsabilidades distintas. Hoje, o usuário interage com CLI ou API; o daemon orquestra estado e integrações; containerd coordena a execução; e runc, alinhado à OCI Runtime Spec, realiza a criação efetiva do container no nível do processo. Esse arranjo não é apenas um detalhe histórico. Ele mostra que segurança de processo, quando tratada corretamente, precisa viver na extremidade da pilha que controla syscalls e credenciais, enquanto a camada superior trabalha com intenção declarativa e metadados [R16][R17][R19].

No Docker, capabilities já são tratadas como política de runtime, não como detalhe interno opaco. A documentação de `docker run` expõe `--cap-add`, `--cap-drop` e `--privileged`, além de listar o conjunto de capabilities mantido por padrão em containers Linux. Esse desenho tem dois efeitos importantes para o seu projeto. Primeiro, ele normaliza a ideia de que privileges são modelados como dados. Segundo, ele mostra que o produto precisa ter defaults explícitos: um runtime maduro não exige que cada workload declare manualmente tudo; ele oferece um perfil padrão pequeno e compreensível, e qualquer ampliação passa a ser uma exceção consciente [R12][R13].

O mesmo vale para `no_new_privs`. O Docker expõe a opção `--security-opt no-new-privileges=true`, e a própria documentação observa que, com ela, comandos como `su` e `sudo` deixam de conseguir elevar privilégio dentro do container. A observação é importante porque traduz a semântica do kernel para uma consequência operacional que o usuário percebe: certas transições de identidade baseadas em `execve()` deixam de funcionar. Em um sistema from scratch, essa semântica não pode ficar escondida. Ela precisa aparecer tanto no modelo de política quanto no material de observabilidade e troubleshooting [R14].

A pilha Docker também é instrutiva na maneira como combina seccomp com capabilities. O perfil default de seccomp é uma allowlist com ação padrão de erro, e a documentação do `docker run` afirma que esse perfil se ajusta às capabilities selecionadas para permitir o uso de facilidades que de fato tenham sido concedidas. Essa relação é mais sofisticada do que simplesmente “aplique capabilities” e “aplique seccomp” em paralelo. O insight de arquitetura é que ambos devem ser compilados a partir de uma mesma intenção de segurança. Em outras palavras, o seccomp profile ideal não é um apêndice arbitrário do container

spec; ele é a projeção syscall-level do capability budget e do tipo de workload [R13] [R15].

Na OCI Runtime Spec, essa integração fica visível no formato da especificação. A seção process contempla capabilities separadas em effective, bounding, inheritable, permitted e ambient, além de noNewPrivileges; a seção linux contempla seccomp. Já no código de configuração do runc, encontram-se estruturas correspondentes para capabilities, seccomp e NoNewPrivileges. Essa equivalência entre especificação declarativa e estrutura interna de runtime deveria inspirar diretamente o desenho do seu projeto. O control plane em Python pode produzir um objeto no estilo OCI; o runtime C++ pode consumi-lo como um plano de execução tipado, sem precisar entender o contexto completo do produto [R17][R18] [R19].

| Na pilha Docker/OCI                                   | No projeto Python + Cython + C++                            | Leitura arquitetural                                                                  |
|-------------------------------------------------------|-------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>CLI/API e daemon do Docker</b>                     | API, metadata store e control plane em Python               | Camada onde intenção, defaults, persistência e experiência de produto são definidos.  |
| <b>containerd / transformação em spec</b>             | Compilador de política e `LaunchSecurityPlan` em Python     | Camada que converte intenção em contrato declarativo executável.                      |
| <b>OCI `process.capabilities` e `noNewPrivileges`</b> | Value objects de domínio e DTOs tipados atravessando Cython | O vocabulário de segurança precisa ser estável e não pode depender de strings livres. |
| <b>OCI `linux.seccomp`</b>                            | Perfis seccomp versionados por arquitetura                  | Seccomp deve ser tratado como artefato de produto, não como texto ad hoc.             |
| <b>runc / libcontainer</b>                            | Runtime C++ de execução privilegiada e `execve()`           | Camada que materializa a política em syscalls, libcap, prctl e libseccomp.            |

*Tabela 2 — Equivalência entre a pilha Docker/OCI e o desenho proposto*

A principal contextualização para o seu sistema, portanto, não é “copiar o Docker”, e sim reproduzir a sua boa separação de responsabilidades. No seu caso, isso significa que Python deve possuir o vocabulário de negócio, o armazenamento de políticas, a API de configuração e o compilador declarativo; Cython deve garantir que essa semântica chegue de forma estável e verificável ao runtime; e C++ deve concentrar a maquinaria irreversível que conversa com kernel, altera credenciais, instala seccomp e finalmente faz `execve()`. Qualquer tentativa de diluir essa responsabilidade entre camadas produzirá exatamente o tipo de ambiguidade que sistemas de segurança não toleram [R17][R19].



## 4. Proposta de System Design para o projeto

O desenho mais robusto para essa feature parte de um princípio simples: a política de segurança é um dado de domínio, mas a sua aplicação é um protocolo de runtime. Isso implica separar o sistema em quatro blocos lógicos. O primeiro é o control plane em Python, responsável por receber a intenção do usuário, resolver defaults, compor perfis, validar regras de negócio e persistir metadados. O segundo é um compilador de política, também em Python, que transforma especificações de alto nível em um LaunchSecurityPlan imutável. O terceiro é a fronteira Cython, cujo papel não é reinterpretar a política, e sim serializá-la de forma tipada, com versão de schema, checksum e semântica estável. O quarto é o runtime C++, um componente de baixo nível, preferencialmente executando em processo próprio, que recebe esse plano, realiza a sequência privilegiada e produz um resultado auditável.

No control plane, a feature deveria aparecer como parte de um modelo maior de workload. Em vez de expor diretamente chamadas do kernel, a API deveria trabalhar com conceitos como `process_identity`, `capability_profile`, `exec_security`, `seccomp_profile`, `compatibility_mode` e `observability_policy`. A persistência não deve guardar apenas o pedido bruto do usuário. Ela deve guardar também o perfil resolvido, a versão dos defaults usados, a arquitetura alvo, a versão de kernel contra a qual o perfil foi validado, o digest do plano compilado e o snapshot observado do processo após o endurecimento. Essa trilha torna o comportamento reproduzível e explicável.

O compilador de política é o coração do desenho em Python. É ele quem deve transformar uma combinação de imagem, comando, modo de compatibilidade e perfil de segurança em um artefato fechado. Esse artefato não é apenas um espelho das flags de entrada. Ele precisa conter invariantes já resolvidos, como a relação entre capability sets, a decisão explícita sobre `no_new_privs`, o perfil seccomp final por arquitetura, e um orçamento temporário de setup. Esse último ponto merece ênfase: o workload final talvez precise de zero capabilities, mas o launcher pode precisar, por alguns microssegundos, de `CAP_SETPCAP`, `CAP_SETUID`, `CAP_SETGID` e talvez outras capacidades para concluir o preparo. O plano, portanto, deve distinguir entre “setup budget” e “final budget”, sendo apenas o segundo uma propriedade visível do workload.

A fronteira Cython deve ser tratada como um anti-corruption layer. O vocabulário de Python, rico em objetos de domínio, datas, enums e políticas versionadas, não deve vazar como JSON frouxo para o C++. Em vez disso, vale adotar structs ou arrays tipados, enums numéricos estáveis para capabilities e ações seccomp, e buffers imutáveis para listas de regras. O lado Python deveria validar tudo antes de cruzar a fronteira; o lado C++ deveria assumir que recebe um plano já coerente, mas ainda assim revalidar invariantes críticas antes de iniciar operações irreversíveis. Esse duplo cheque é barato comparado ao custo de um bug de segurança e reduz a superfície de inconsistência entre linguagem e runtime.

A decisão de manter o runtime C++ em processo próprio merece ser registrada como recomendação forte, não opcional. Se a sequência sensível de fork/clone,

troca de credenciais, `prctl`, `libcap` e `libseccomp` ocorrer dentro do mesmo processo do interpretador Python, o projeto herda problemas de fork-safety, interação com threads, estado global de bibliotecas e complexidade de depuração. O desenho mais limpo é um pequeno supervisor C++ acionado por Unix socket ou por invocação controlada, responsável por criar o processo filho de container, executar o protocolo de segurança e reportar o resultado ao control plane. Esse é um paralelo direto com a separação Docker/containerd/runc, adaptado à sua stack [R16][R19].

No lado do runtime, o fluxo ideal contém um processo pai de supervisão e um processo filho de preparação. O pai recebe o plano, abre canais de comunicação para status e erro, e cria o filho. O filho executa o preparo do ambiente que exige privilégios — namespaces, mounts, root filesystem, descritores, limites e qualquer outra etapa já prevista pelo runtime — e somente ao final inicia a transição de segurança propriamente dita. Essa separação ajuda a manter o caminho até o `execve()` curto, sequencial e verificável. Também facilita a emissão de eventos de observabilidade para o control plane, como “prepare\_begin”, “capabilities\_converged”, “bounding\_dropped”, “nnp\_set”, “seccomp\_loaded” e “exec\_started”.

Um bom system design também precisa prever introspecção. O runtime C++ deveria ler `/proc/self/status` imediatamente antes do `execve()` — ou executar um binário de prova em ambiente de teste — para capturar `CapEff`, `CapPrm`, `CapInh`, `CapBnd`, `CapAmb`, `NoNewPrivs`, `Seccomp` e `Seccomp_filters`. Esses campos são o elo entre a política declarada e o estado real do kernel. O snapshot, assinado pelo digest do plano, pode retornar ao control plane como parte do registro de execução. Com isso, o sistema deixa de operar em “confiança implícita no launcher” e passa a operar em “verificação observável do launcher” [R11].

Por fim, o system design deve assumir uma postura fail-closed desde a primeira linha. Se `cap_set_proc()` falhar, se `cap_drop_bound()` falhar, se `prctl(PR_SET_NO_NEW_PRIVS)` falhar, se `seccomp_load()` falhar ou se a verificação observada divergir do plano compilado, a criação do container precisa falhar integralmente. Não existe estado intermediário aceitável em que o workload inicie “com parte da política aplicada”. Segurança de processo é um protocolo de tudo ou nada, e a arquitetura precisa refletir isso desde o contrato da API até a semântica de retorno do runtime [R2][R6][R7][R10].

## Arquitetura da feature

Policy declarativa em Python, contrato estável em Cython e execução irreversível no runtime C++.

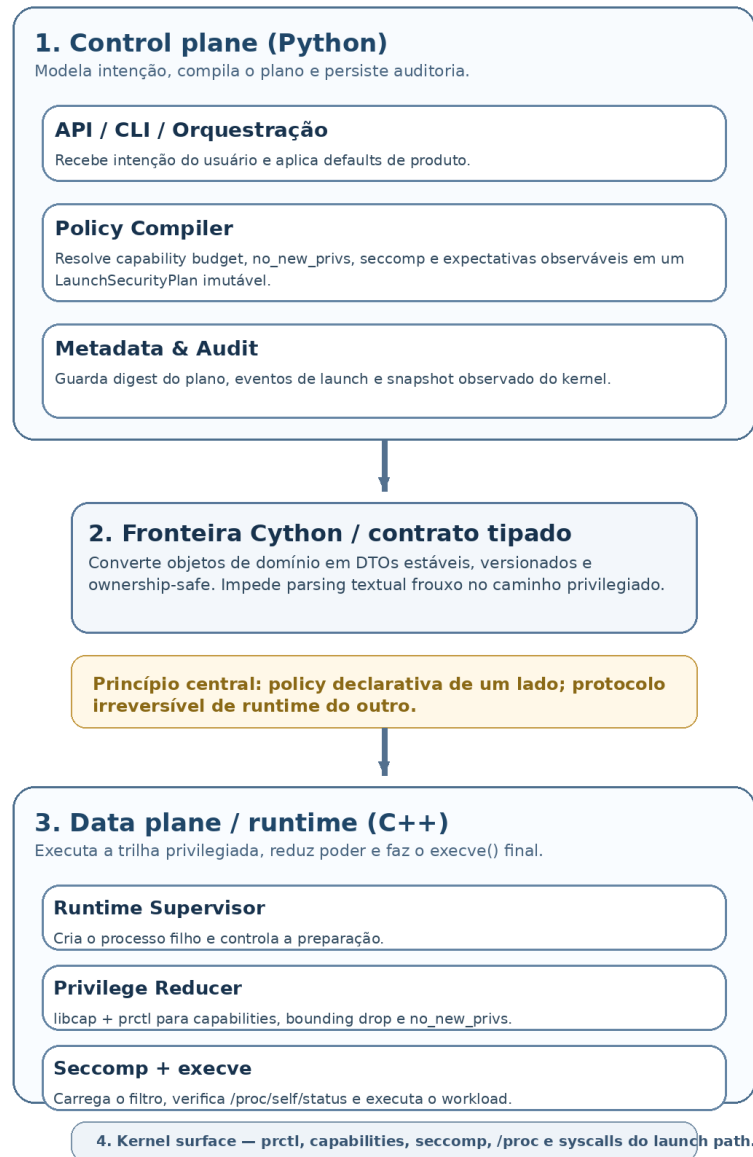


Figura 1 — Separação entre policy declarativa, contrato Cython e protocolo irreversível de runtime.

## 5. Low Level Design do launcher e do protocolo pré-exec

O low level design deve começar por uma decisão estrutural: a trilha de criação do processo precisa permanecer single-threaded a partir do momento em que entra na fase de endurecimento. Capabilities e bounding set têm semântica por thread, e esse detalhe torna arriscado qualquer desenho que permita concorrência interna no momento de aplicar a política. A forma mais segura de implementar o launcher é tratar o filho que fará `execve()` como um pequeno autômato sequencial, sem criação de threads, sem dependência de estado externo mutável e com uma taxonomia de erros inteiramente explícita [R9][R11].

No plano de dados, o runtime deveria trabalhar com quatro value objects centrais. `ProcessIdentity` representa UID, GID e grupos suplementares finais. `CapabilityProfile` representa os cinco conjuntos relevantes, mas o MVP pode impor invariantes mais fortes do que o kernel impõe, por exemplo exigindo que `effective  $\subseteq$  permitted  $\subseteq$  bounding` e que `ambient  $\subseteq$  inheritable  $\cap$  permitted`, com `ambient=[]` por padrão. `SeccompProfile` representa a ação padrão, a lista de regras e a arquitetura a que o perfil se refere. `LaunchSecurityPlan`, por sua vez, agrega identidade, orçamento temporário de setup, capability profile final, política de `no_new_privs`, seccomp profile, expectativas de observabilidade e digest de compilação.

Do ponto de vista de código nativo, vale encapsular a interação com o kernel em pequenos adapters orientados a responsabilidade. Um `PrctlAdapter` cobre `PR_SET_NO_NEW_PRIVS` e `PR_CAPBSET_DROP`. Um `LibcapAdapter` encapsula `cap_get_proc`, `cap_set_proc`, limpeza de `ambient capabilities` e operações de bounding via libcap. Um `SeccompAdapter` encapsula `seccomp_init`, `seccomp_rule_add`, `seccomp_load` e `seccomp_release`. Um `ProcStatusProbe` lê `/proc/self/status` e materializa uma struct de observação. Essa decomposição evita que o método de launch se torne um bloco monolítico de syscalls e ifs, o que é ruim tanto para legibilidade quanto para auditabilidade.

A sequência operacional recomendada é a seguinte. O processo filho entra com o orçamento temporário de setup, executa toda preparação que ainda demande capabilities elevadas, estabiliza descritores e root filesystem, resolve a identidade final e, em seguida, começa a transição para o orçamento final. Primeiro, limpa explicitamente `ambient capabilities`, porque o objetivo de um runtime previsível é não carregar privilégio implícito por ambiente através de `execve()`. Depois, aplica com libcap os conjuntos efetivo, permitido e herdável que deverão valer ao final. Nesse ponto, o launcher ainda preserva `CAP_SETPCAP` apenas pelo tempo necessário para dropar o bounding set remanescente. Concluída essa etapa, realiza a mudança de identidade final quando necessário, marca `no_new_privs`, instala o filtro seccomp e só então chama `execve()` [R1][R3][R9][R10].

```
# Pseudocódigo simplificado da trilha segura de launch
child_main(plan):
    assert_single_threaded()

    prepare_namespaces_mounts_cgroups_rootfs(plan.setup_context)
    configure_stdio_rlimits_hostname(plan.runtime_context)

    clear_ambient_capabilities()

    apply_capability_sets(
        effective=plan.final_caps.effective,
        permitted=plan.final_caps.permitted,
        inheritable=plan.final_caps.inheritable
    )

    keep_CAP_SETPCAP_temporarily()
    for cap in all_caps_not_in(plan.final_caps.bounding):
        drop_from_bounding_set(cap)

    switch_identity(plan.identity)    # quando suportado pelo modo selecionado
    prctl(PR_SET_NO_NEW_PRIVS, 1)
```

```
seccomp_ctx = seccomp_init(plan.seccomp.default_action)
for rule in plan.seccomp.rules:
    seccomp_rule_add(seccomp_ctx, rule)
seccomp_load(seccomp_ctx)

observed = read_proc_status()
assert_matches(plan.expected_state, observed)

execve(plan.argv[0], plan.argv, plan.envp)
```

Há duas sutilezas de projeto embutidas nessa ordem. A primeira é que o bounding set deve ser tratado como uma cerca irreversível do futuro, não como um substituto para a limpeza imediata dos conjuntos correntes. Mesmo depois de um `PR_CAPBSET_DROP`, o launcher ainda precisa garantir que o processo não retenha bits efetivos ou permitidos indevidos. A segunda é que o runtime não deveria misturar, na mesma entidade de domínio, capabilities necessárias para preparar o ambiente e capabilities que o workload final merece manter. O orçamento temporário de setup é um detalhe do mecanismo; o orçamento final é um detalhe do domínio do workload. Misturar os dois conceitos tende a vazar complexidade operacional para a API e cria superfícies de erro em que o usuário solicita uma capability “porque sem ela o launcher não funciona”, quando na verdade quem precisa dela é apenas a fase transitória do runtime.

Para o MVP, convém restringir explicitamente os casos aceitos. A rota mais segura é suportar integralmente dois cenários: workload executando como usuário final sem capabilities residuais, ou workload executando com um conjunto mínimo de capabilities finais ainda como UID privilegiado apenas quando estritamente justificado pelo produto. O caso intermediário — trocar para um UID não privilegiado e ao mesmo tempo reter capabilities específicas depois da troca — é possível em Linux, mas adiciona complexidade relevante à ordem de operações e ao tratamento de credenciais. Em vez de fingir completude, o projeto deveria registrar isso como fase avançada e bloquear a configuração no compilador de política até que exista cobertura de teste apropriada.

O `SeccompProfile` merece também um desenho de dados cuidadoso. Em vez de armazenar regras como strings arbitrárias do mundo Docker/OCI, é melhor compor uma AST mínima em Python e convertê-la em estruturas estáveis antes da passagem para C++. Cada regra pode carregar syscall, ação, comparadores de argumentos e anotações de compatibilidade por arquitetura. Essa modelagem simplifica validações, evita parsing textual no caminho privilegiado e permite gerar artefatos de auditoria. Como a `libseccomp` já oferece semântica relativamente portátil para `seccomp_rule_add()`, o projeto ganha robustez ao centralizar a tradução em um único componente [R6][R7][R8].

No nível da memória e da FFI, a regra deveria ser imutabilidade. O objeto Python compilado deve ser congelado, convertido por Cython em buffers e structs ownership-safe, e entregue ao C++ sem necessidade de round-trips. O Cython deveria liberar o GIL durante a chamada ao runtime e mapear a taxonomia de erro

nativa para exceções Python estáveis, preservando erro, etapa de falha e operação correspondente. Isso transforma um erro nativo obscuro em uma informação útil para troubleshooting, algo crítico quando o problema é “por que este container não subiu sob uma política mais restrita?”.

Outro ponto de low level design, muitas vezes negligenciado, é a verificação ativa do resultado. Não basta presumir que uma sequência de chamadas bem-sucedidas produziu o estado desejado. O launcher deve ter uma etapa explícita de `assert_observed_state`, que compara o snapshot de `/proc/self/status` com o estado esperado pelo plano. Em produção, essa etapa pode ser resumida a campos essenciais; em testes e em modo de debug, ela pode ser mais extensa. Esse passo não elimina a necessidade de confiança no kernel, mas elimina a confiança cega na implementação do launcher.

Uma implementação madura também deve decidir cedo como tratar perfis seccomp em rollout. Uma estratégia eficaz é possuir pelo menos três modos. O primeiro é compatível, com `SCMP_ACT_ERRNO` e `allowlist` mais ampla. O segundo é endurecido, também em `allowlist`, mas com regras mais estreitas e possibilidade de `SCMP_ACT_KILL_PROCESS` para chamadas inaceitáveis. O terceiro é de observação controlada, usando logging ou perfil mais permissivo em ambientes de staging para descobrir syscalls ainda necessárias. O produto não precisa expor todos esses modos ao usuário final desde o primeiro dia, mas a arquitetura deve acomodá-los sem reescrita [R6][R7][R15].

Finalmente, o launcher precisa ser desenhado como código que aceita a irreversibilidade como vantagem. `Bounding drop`, `no_new_privs` e `seccomp` convertem o processo em algo progressivamente menos poderoso. Essa natureza monotônica permite um design particularmente limpo: cada etapa só pode manter ou reduzir poder; nunca aumentá-lo. Quando o código é escrito para respeitar essa monotonicidade, fica mais fácil provar que não há “atalhos” de privilégio entre o início da preparação e o `execve()`. Esse é o tipo de propriedade que diferencia um runtime meramente funcional de um runtime confiável.



## Protocolo pré-exec recomendado

Cada etapa só mantém ou reduz poder. Qualquer falha deve abortar o launch.



Figura 2 — Ordem recomendada de endurecimento imediatamente antes do `execve()`.

## 6. Domain-Driven Design aplicado à feature

Aplicar Domain-Driven Design a uma feature tão próxima do kernel pode parecer excessivo à primeira vista, mas na prática é o contrário. Quanto mais baixo o nível técnico, mais importante se torna preservar um modelo conceitual claro no alto nível do produto. Sem isso, o control plane vira apenas um repasse de flags para syscalls, e o sistema perde a capacidade de validar intenção, explicar comportamento e evoluir sem vazamento de detalhes de implementação. O papel do DDD aqui é impedir que a API do produto seja capturada pela acidentalidade do kernel.

A linguagem ubíqua do domínio deveria partir de poucos termos com semântica rígida. `Workload` é a unidade executável do sistema. `SecurityPolicy` é a intenção declarativa de segurança para um workload. `CapabilityBudget` é o conjunto de privileges que um processo pode exercer, distinguindo explicitamente orçamento de setup e orçamento final. `ExecutionIdentity` define quem o processo será quando começar a executar o workload. `SeccompProfile` define o envelope syscall-level permitido. `LaunchSecurityPlan` é a materialização determinística dessa política para um lançamento concreto. `ObservedSecurityState` é a leitura do estado real retornado pelo kernel. Esses termos permitem conversas de produto, de engenharia e de operação sobre o mesmo objeto sem confusão semântica.

Em termos de bounded contexts, há pelo menos quatro. O contexto de `Policy Authoring` vive no control plane e trata de defaults, presets, perfis e compatibilidade. O contexto de `Launch Compilation` traduz essas intenções em um plano fechado, validando invariantes e escolhendo perfis concretos por arquitetura. O contexto de `Runtime Execution` pertence ao C++ e cuida da máquina de estados do processo até o `execve()`. O contexto de `Security Audit` trata da observação do estado aplicado, da persistência de snapshots e do suporte à investigação. `Cython` atua como uma anti-corruption layer entre `Launch Compilation` e `Runtime Execution`: ele impede que o modelo rico de Python vaze, desestruturado, para o lado nativo.

No modelo tático, `SecurityPolicy` e `LaunchSecurityPlan` são os agregados mais importantes. `SecurityPolicy` concentra invariantes de negócio, como “o modo padrão exige `no_new_privs=true`”, “ambient capabilities são proibidas no perfil baseline”, “capabilities finais precisam ser subconjunto do catálogo permitido pela plataforma” e “um workload que pede compatibilidade máxima não pode receber o perfil `seccomp` mais endurecido sem `opt-in` explícito”. `LaunchSecurityPlan` concentra invariantes de execução, como “o digest precisa casar com a versão do compilador”, “o setup budget não pode ser menor do que as operações de setup planejadas” e “o estado esperado observável precisa ser calculável a partir do plano”.

Value objects são particularmente valiosos neste domínio porque praticamente tudo que importa aqui é imutável por natureza. `CapabilitySet`, `ExecutionIdentity`, `SeccompRule`, `SeccompArchitecture`, `KernelFeatureMatrix` e `ProcStatusSnapshot` são bons candidatos. Eles não deveriam carregar lógica de orquestração pesada, mas deveriam carregar validações locais e comparações semânticas. Um `CapabilitySet`, por exemplo, pode saber responder se é subconjunto de outro, se respeita o baseline do produto e se viola invariantes deliberadamente mais estritas do que as do kernel.

Serviços de domínio entram quando a regra deixa de caber em um único agregado. `PolicyCompiler` compõe policy authoring com compatibilidade de plataforma. `CapabilityResolver` calcula os conjuntos efetivo, permitido, herdável, ambiente e bounding a partir da intenção declarada. `SeccompCompiler` transforma um perfil lógico em regras concretas por arquitetura. `LaunchPlanValidator` executa a verificação final antes do plano cruzar a fronteira `Cython`. Nenhum desses serviços deveria depender diretamente de syscalls; seu papel é manter o domínio limpo e previsível.

Eventos de domínio também são úteis. `SecurityPlanCompiled`, `RuntimeLaunchStarted`, `PrivilegesReduced`, `NoNewPrivilegesSet`, `SeccompInstalled`, `ObservedStateRecorded` e `LaunchAborted` formam uma narrativa operacional clara. Além de suportar auditoria, esses eventos ajudam a separar o “o que o sistema decidiu” do “o que o kernel efetivamente aceitou”. Em um produto sério, essa distinção importa tanto para depuração quanto para conformidade.

O ganho prático do DDD, portanto, não é acadêmico. Ele produz uma API mais compreensível, uma persistência mais rica, um runtime menos acoplado ao resto do produto e um caminho de evolução mais limpo. Quando, no futuro, o projeto incorporar user namespaces, perfis LSM ou mecanismos adicionais como Landlock, esses novos recursos poderão entrar como extensões de domínio, e não como enxertos desordenados sobre um contrato já quebrado.

## 7. Padrões de Design (GoF) recomendados

Uma feature de segurança de processo tende a deteriorar rapidamente quando a implementação é apenas procedural. O código passa a acumular condicionais por kernel, branches por perfil, pequenas exceções por workload e uma sequência difícil de auditar de chamadas a bibliotecas nativas. Padrões clássicos de design não resolvem o problema sozinhos, mas ajudam a preservar legibilidade, isolamento de responsabilidade e capacidade de teste. Neste contexto, eles devem ser usados como instrumentos de redução de acoplamento, não como ornamentação acadêmica.

O padrão Builder é particularmente adequado para a montagem do `LaunchSecurityPlan`. A política chega ao compilador em partes: identidade de execução, perfil base da plataforma, escolhas do usuário, modo de compatibilidade, imagem, arquitetura e talvez requisitos do executor. Um builder permite construir esse plano em etapas, validando invariantes à medida que os dados entram e impedindo a criação de um plano parcialmente válido. Ele também é útil para separar o plano externo visível na API do plano interno já enriquecido com setup budget e estado esperado.

Strategy é a melhor escolha para seleção de perfis. Diferentes workloads e diferentes plataformas exigirão estratégias distintas para capabilities e seccomp. Um `CapabilityResolutionStrategy` pode decidir como sair de uma policy abstrata para um budget concreto. Um `SeccompCompilationStrategy` pode escolher entre baseline, compatibilidade ampliada ou endurecimento. Como o Docker já sugere, inclusive, que o perfil seccomp se ajusta às capabilities escolhidas, esse padrão ajuda a preservar a relação entre as duas decisões sem espalhar ifs pelo código [R13][R15].

No lado C++, Facade e Adapter são úteis para esconder detalhes sujos de bibliotecas e syscalls. O restante do runtime não deveria conhecer diferenças miúdas entre `prctl`, `libcap` e `libseccomp`. Ele deveria conversar com interfaces pequenas como `CapabilityController`, `NoNewPrivsController` e `SeccompController`. Isso torna testes mais simples, facilita instrumentação e permite trocar detalhes internos sem reescrever o fluxo de launch.

O padrão Command encaixa bem na execução da trilha privilegiada. Cada etapa irreversível pode ser um comando explícito: PrepareFilesystem, ConvergeCapabilities, DropBoundingSet, SetNoNewPrivs, LoadSeccomp, AssertObservedState e ExecWorkload. O benefício não é apenas estético. Com comandos explícitos, o runtime consegue registrar a etapa atual, mapear falhas com precisão e montar uma trilha de auditoria operacional muito melhor.

Template Method ajuda a fixar a ordem das operações. O launch seguro tem um esqueleto rígido: preparar, convergir, reduzir, marcar, filtrar, verificar, executar. Subclasses ou políticas específicas podem alterar detalhes de alguns passos, mas não deveriam poder inverter a ordem fundamental sem um opt-in muito consciente. Esse padrão reforça a monotonicidade de poder ao longo da execução.

State também é valioso porque a feature é, essencialmente, uma máquina de estados. Antes do preparo, o processo está em Created; depois, em Prepared; depois, em CapabilitiesConverged; depois, em BoundingReduced; depois, em NoNewPrivsLocked; depois, em SeccompLocked; por fim, em Executed ou Failed. Modelar isso explicitamente ajuda a impedir transições ilegais e a construir telemetria coerente.

| Padrão                  | Aplicação concreta                                                                                   | Benefício principal                                                      |
|-------------------------|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>Builder</b>          | Montagem progressiva do `LaunchSecurityPlan` a partir de policy, compatibilidade e defaults.         | Impede criação de planos parciais e centraliza validação de invariantes. |
| <b>Strategy</b>         | Seleção de `CapabilityResolutionStrategy` e `SeccompCompilationStrategy` por workload e arquitetura. | Evita ifs espalhados e permite presets de segurança evolutivos.          |
| <b>Facade / Adapter</b> | Encapsulamento de `prctl`, libcap, libseccomp e leitura de `/proc`.                                  | Reduz acoplamento do runtime com detalhes de syscall e biblioteca.       |
| <b>Command</b>          | Representação explícita de passos como `DropBoundingSet` e `LoadSeccomp`.                            | Melhora auditoria, rollback lógico por abort e precisão de erro.         |
| <b>Template Method</b>  | Fixação da ordem preparar → convergir → reduzir → marcar → filtrar → executar.                       | Preserva a monotonicidade de privilégio ao longo do launch.              |
| <b>State</b>            | Modelagem dos estados `Prepared`, `CapabilitiesConverged`, `NoNewPrivsLocked`, `SeccompLocked`.      | Impede transições ilegais e melhora telemetria do runtime.               |

Tabela 3 — Padrões GoF sugeridos e seu papel na implementação

A combinação desses padrões produz um efeito arquitetural importante. O domínio permanece descritivo e declarativo; o compilador permanece determinístico; a fronteira Cython permanece simples; e o runtime nativo continua pequeno, sequencial e auditável. Esse é exatamente o tipo de disciplina estrutural que um projeto from scratch precisa para não reproduzir, em código próprio, a mesma complexidade accidental que o ecossistema Docker levou anos para domesticar.

## 8. Requisitos funcionais e não funcionais

Os requisitos desta feature precisam refletir uma característica central do problema: não basta “suportar” mecanismos do kernel; é preciso suportá-los de maneira observável, determinística e segura por padrão. A melhor forma de expressar isso é separar requisitos funcionais, que descrevem o que o sistema deve fazer, de requisitos não funcionais, que descrevem como esse comportamento precisa se manifestar para ser confiável em produção.

### 8.1 Requisitos funcionais

**RF-01** — O sistema deve compilar, a partir de uma `SecurityPolicy` declarativa, um `LaunchSecurityPlan` imutável, versionado e reproduzível. Isso significa que duas entradas semanticamente idênticas, executadas na mesma matriz de compatibilidade, devem produzir o mesmo plano, inclusive no que diz respeito a capability sets, `no_new_privs`, `seccomp` e expectativas de observabilidade. O critério de aceite é a estabilidade do digest do plano e a possibilidade de persisti-lo no metadata store.

**RF-02** — O compilador de política deve validar invariantes fortes antes que qualquer execução aconteça. Entre elas, o projeto deve rejeitar `ambient capabilities` no perfil baseline, exigir coerência entre capability sets, impedir perfis `seccomp` incompatíveis com a arquitetura alvo e recusar combinações que o MVP ainda não suporta, como retenção de capabilities por processo não privilegiado sem a trilha avançada correspondente. O critério de aceite é a falha antecipada, em control plane, para qualquer plano semanticamente inseguro ou não suportado.

**RF-03** — O runtime C++ deve ser capaz de aplicar capabilities usando `libcap`, convergindo explicitamente os conjuntos efetivo, permitido e herdável antes do `execve()`, e limpando capacidades ambiente quando a política assim exigir. O critério de aceite é a correspondência entre o plano compilado e os campos `CapEff`, `CapPrm`, `CapInh` e `CapAmb` observados no processo alvo [R10][R11].

**RF-04** — O runtime deve ser capaz de dropar o capability bounding set de forma irreversível antes do `execve()`, preservando `CAP_SETPCAP` apenas pelo intervalo mínimo necessário para executar a operação. O critério de aceite é que `CapBnd` observado seja subconjunto exato do bounding previsto no plano e que tentativas posteriores de recuperar bits removidos sejam impossíveis [R9][R11].

**RF-05** — O runtime deve aplicar `PR_SET_NO_NEW_PRIVS` antes da instalação do filtro `seccomp` e antes do `execve()` final. O critério de aceite é observar `NoNewPrivs: 1` em

`/proc/self/status` e demonstrar, por testes de integração, que binários `setuid` ou `file capabilities` não elevam privilégio quando executados sob essa política [R1][R2][R11].

**RF-06** — O sistema deve suportar um perfil `seccomp` básico versionado por arquitetura, compilado a partir de uma representação tipada e carregado via `libseccomp`. O critério de aceite é observar `Seccomp: 2` e, quando possível, `Seccomp_filters > 0`, além de testes negativos demonstrando que syscalls explicitamente proibidas falham conforme a ação configurada [R3][R6][R7][R11].

**RF-07** — O runtime deve operar em `fail-closed`. Se qualquer etapa irreversível falhar, o workload não pode ser iniciado. O critério de aceite é que falhas em `cap_set_proc`, `cap_drop_bound`, `prctl`, `seccomp_load` ou validação observada resultem em `abort` de `launch` com erro estruturado e sem processo de workload em execução.

**RF-08** — O sistema deve registrar, no metadata store, não apenas a política solicitada, mas também o plano compilado, sua versão, o resultado da execução e o snapshot observado do estado de segurança. O critério de aceite é a existência de trilha completa para auditoria e troubleshooting.

**RF-09** — O control plane deve oferecer um modo de pré-validação ou `dry-run` capaz de mostrar ao usuário o capability budget final, o bounding set previsto, o estado de `no_new_privs` e o perfil `seccomp` selecionado sem executar o workload. O critério de aceite é a possibilidade de inspeção e revisão humana antes da execução.

**RF-10** — O produto deve suportar presets de segurança no nível de domínio, como `baseline`, `compatibilidade ampliada` e `endurecido`, sem expor o usuário diretamente às minúcias do kernel em todos os casos. O critério de aceite é a disponibilidade de perfis consistentes, documentados e internamente compilados para planos concretos.

**RF-11** — O runtime deve publicar eventos estruturados de progresso e falha ao longo da trilha de `launch`, permitindo ao control plane reconstruir a linha do tempo da transição de privilégios. O critério de aceite é a emissão de eventos com etapa, `timestamp`, `digest` do plano e código de retorno.

**RF-12** — O sistema deve oferecer uma suíte de testes de integração com binários de prova, cobrindo ao menos: observação de capability sets, verificação de `no_new_privs`, tentativa de uso de binário `setuid`, tentativa de obtenção de `file capabilities` adicionais e execução de syscalls bloqueadas por `seccomp`. O critério de aceite é a automação desses cenários no CI da plataforma.

## 8.2 Requisitos não funcionais

**RNF-01** — A feature deve ser segura por padrão. O perfil default precisa favorecer mínima autoridade, `no_new_privs=true`, `ambient=[]`, `inheritable=[]` e `seccomp` ativo sempre que a plataforma suportar. O sistema só pode relaxar esse baseline por decisão explícita de política.

**RNF-02** — O comportamento deve ser determinístico e reproduzível. O mesmo plano compilado, na mesma matriz de kernel e arquitetura suportada, deve



produzir o mesmo envelope de segurança observável. Isso é essencial para depuração, auditoria e regressão.

**RNF-03** — A implementação deve minimizar a superfície de código privilegiado. O caminho que ainda opera antes da queda de privilégios deve residir em um runtime C++ pequeno, especializado e auditável, em vez de ficar espalhado pelo control plane em Python.

**RNF-04** — O caminho crítico de endurecimento deve ser single-threaded e livre de concorrência interna. Isso reduz ambiguidade sobre semântica por thread de capabilities e simplifica a prova informal de correção.

**RNF-05** — A solução deve ser observável. Logs, métricas e eventos precisam registrar plano, etapa, erro, errno, snapshot observado e tempo gasto por fase, sem vaziar dados desnecessários do workload.

**RNF-06** — A solução deve ser compatível com evolução de kernel e arquitetura por meio de versionamento explícito de perfis e de uma matriz de suporte. Perfis seccomp e defaults não podem ser tratados como constantes eternas.

**RNF-07** — A solução deve degradar com clareza. Quando o kernel não suportar um recurso exigido pelo perfil ou quando a política entrar em uma combinação ainda não implementada, o erro precisa ser explícito e explicável; nunca silencioso.

**RNF-08** — A solução deve preservar desempenho previsível. O overhead introduzido pelo endurecimento precisa ser pequeno em relação ao custo total de criação do container e, sobretudo, estável o bastante para não destruir SLOs de launch.

**RNF-09** — O contrato entre Python, Cython e C++ deve ser estável, versionado e backward compatible dentro de uma janela definida pelo produto. Mudanças de enumeração, semântica ou layout de structs não podem ocorrer de forma implícita.

**RNF-10** — A solução deve ser extensível. A introdução futura de user namespaces, LSMs, perfis Landlock, rootless mode ou seccomp mais sofisticado não deve exigir reescrever o vocabulário principal do domínio.

**RNF-11** — A solução deve ser testável em múltiplos níveis. Serviços de domínio precisam ter testes unitários; o compilador, testes determinísticos; e o runtime, testes de integração que observem o estado real do kernel.

**RNF-12** — A solução deve favorecer análise forense posterior. O sistema precisa manter histórico suficiente para responder, após o fato, qual política foi solicitada, qual plano foi compilado, qual estado foi observado e em que etapa uma falha ocorreu.

## 9. Plano de implementação incremental

Um bom roadmap para essa feature não deve perseguir imediatamente toda a flexibilidade do Docker. Ele deve perseguir uma sequência de incrementos em que cada passo produza uma propriedade de segurança comprovável. A primeira fase é inteiramente de modelagem e contrato. Nela, o control plane em Python ganha os

objetos de domínio, o compilador de política, o `LaunchSecurityPlan`, o versionamento de schema e a validação forte de invariantes. O resultado esperado dessa fase não é um container mais seguro em execução, e sim a capacidade de dizer, de forma determinística, qual envelope de segurança o sistema pretende aplicar.

A segunda fase é a do capability path. O runtime C++ passa a receber o plano compilado e implementa a convergência dos capability sets com `libcap`, a limpeza de `ambient capabilities` e a redução do bounding set. O foco aqui deve ser menos “quantos casos suportamos” e mais “quão observável é o estado resultante”. Ao final dessa fase, o projeto já deve possuir uma suíte de testes que verifica os campos de `/proc/self/status` para workloads de prova e uma trilha de auditoria que retorna esse snapshot ao control plane [R10][R11].

A terceira fase incorpora `no_new_privs` e `seccomp` básico. A recomendação é ativar primeiro um perfil baseline compatível, com allowlist prudente e ação padrão de erro, para reduzir choque de compatibilidade. O rollout ideal começa em ambientes controlados com telemetria forte, passa por perfis de observação para entender syscalls ainda necessárias e só então endurece defaults. O ganho dessa fase é qualitativo: a segurança deixa de ser apenas “menos capabilities” e passa a ser também “menos superfície de kernel” [R3][R6][R7][R15].

A quarta fase é a da maturidade operacional. Nela entram dry-run, eventos de launch, relatórios de divergência entre plano e estado observado, ferramentas de troubleshooting e presets de produto. É nessa etapa que a feature deixa de ser exclusivamente engenharia de runtime e passa a ser uma capacidade usável pelo restante da plataforma. Um time de aplicação ou de operações deve conseguir entender, sem ler código nativo, por que um workload foi recusado ou por que um syscall passou a falhar após endurecimento.

A quinta fase pode atacar flexibilidade avançada: preservação controlada de capabilities em cenários mais complexos, integração com user namespaces e rootless mode, perfis `seccomp` especializados por classe de workload, e posterior composição com mecanismos adicionais de sandbox. O ponto importante é que nenhuma dessas extensões deveria reabrir o desenho base. Se o modelo de domínio, o contrato entre linguagens e o runtime monotônico estiverem corretos, evoluções futuras serão incrementais; se estiverem errados, cada nova feature de segurança exigirá redesenho do produto.

Essa ordem incremental tem uma virtude estratégica. Ela transforma um problema de segurança amplo e assustador em uma série de propriedades pequenas e verificáveis. Primeiro o sistema sabe declarar; depois sabe aplicar capabilities; depois sabe bloquear ganho de privilégio por `execve()`; depois sabe reduzir syscalls; por fim sabe se explicar. Para um projeto from scratch, essa progressão é muito mais valiosa do que tentar “ter tudo” logo na primeira entrega.

## 10. Estratégia de testes, validação e observabilidade

A feature só estará realmente pronta quando houver evidência observável de que o estado do processo, imediatamente antes do `execve()`, corresponde ao plano compilado. Testes unitários são necessários, mas insuficientes. O projeto precisa de uma estratégia em camadas. No nível de domínio, testes determinísticos devem provar que o compilador produz sempre o mesmo `LaunchSecurityPlan` para a mesma entrada. No nível da fronteira Cython, testes devem garantir estabilidade de layout, serialização e mapeamento de erro. No nível do runtime, testes de integração precisam observar o kernel de verdade.

O binário de prova é uma ferramenta central nesse desenho. Em vez de testar apenas por inferência, o CI pode executar um pequeno programa que imprime os campos relevantes de `/proc/self/status`, tenta executar um binário `setuid`, tenta explorar file capabilities conhecidas e faz chamadas a syscalls deliberadamente proibidas. O resultado desses testes fornece uma assinatura concreta do comportamento do launcher. Em um projeto que lida com segurança de processo, esse tipo de harness vale mais do que dezenas de mocks.

A observabilidade operacional deve acompanhar essa estratégia. Cada launch precisa registrar o digest do plano, a fase em curso, a duração de cada etapa, o resultado observado e a causa de falha quando houver. Isso não é apenas bom para engenharia. Também é crucial para experiência do usuário interno da plataforma, porque endurecimento sempre traz trade-offs de compatibilidade, e esses trade-offs só são administráveis quando o sistema consegue explicar exatamente onde e por que um workload falhou.

Vale incluir também testes negativos específicos para as sutilezas mais perigosas do domínio. Um deles deve provar que dropar o bounding set sem limpar conjuntos correntes não é suficiente, justamente para impedir regressões conceituais. Outro deve provar que carregar `seccomp` sem `no_new_privs` ou sem o privilégio correspondente falha como esperado. Outro deve verificar que o runtime continua fail-closed se a verificação observada divergir do planejado. Esses testes não apenas defendem o código; eles defendem o modelo mental da equipe [R3][R9][R11].

```
CapPrm: 0000000000000000
CapEff: 0000000000000000
CapInh: 0000000000000000
CapBnd: 0000000000000000
CapAmb: 0000000000000000
NoNewPrivs: 1
Seccomp: 2
Seccomp_filters: 1
```

Além do CI, convém ter uma matriz de validação por kernel e arquitetura oficialmente suportados. Perfis `seccomp` e semânticas de determinadas chamadas podem variar ao longo do tempo; portanto, o projeto deveria testar pelo menos o

baseline em todas as combinações suportadas, registrando diferenças conhecidas de comportamento. Sem essa disciplina, o produto corre o risco de transformar “segurança” em “surpresa dependente de ambiente”.

Por fim, a observabilidade deve fechar o ciclo com o domínio. O snapshot observado não deveria ser um artefato descartável de debug; ele deveria alimentar o contexto de Security Audit, ficar associado ao LaunchRecord e poder ser consultado posteriormente. Isso viabiliza não apenas troubleshooting, mas também revisões de política, análise de regressão e aprendizado operacional sobre o que realmente é necessário para diferentes classes de workload.

## 11. Riscos, trade-offs e decisões arquiteturais críticas

O primeiro risco arquitetural é tratar capabilities, no\_new\_privs, bounding set e seccomp como caixas independentes. Essa visão parece modular, mas leva a planos incoerentes. Um exemplo clássico é acreditar que bounding drop sozinho “resolve” privilégio futuro, ignorando conjuntos correntes e ambient capabilities. Outro é instalar seccomp como último detalhe cosmético, sem perceber que sua aplicabilidade e seu valor dependem da forma como o processo já foi reduzido. O desenho precisa partir da ideia de protocolo único de transição de estado, e não de checklist de hardening [R1][R3][R9].

O segundo risco é misturar política de negócio com necessidade mecânica do launcher. Se o usuário pede um workload sem capabilities finais, não faz sentido expor ao usuário capacidades transitórias que só existem porque o runtime precisa terminar o preparo. Quando essa separação não existe, a API fica poluída, o metadata store perde clareza e a segurança fica mais difícil de auditar. O setup budget deve ser interno e derivado; o final budget deve ser externo e explicável.

O terceiro risco está na compatibilidade. Seccomp é poderoso justamente porque é específico o bastante para reduzir superfície de ataque, mas essa especificidade implica sensibilidade a arquitetura, libc e estilo de workload. Um perfil rígido demais quebra aplicações legítimas; um perfil frouxo demais perde valor. A resposta não é abandonar seccomp, e sim versionar perfis, testá-los sistematicamente e oferecer presets de produto. Segurança madura raramente nasce de uma política universal estática; ela nasce de políticas explícitas, testadas e evolutivas.

Há também um trade-off entre paridade imediata e correção inicial. Implementar desde o primeiro dia cenários como retenção de capabilities em processos não privilegiados, rootless mode, integração ampla com namespaces de usuário e perfis avançados por classe de workload pode ser atraente, mas tende a espalhar complexidade antes que a base observável exista. Para um projeto from scratch, a decisão mais racional é começar com um subconjunto estrito e aumentar a expressividade somente quando já houver evidência de que a trilha básica está correta.

Outro ponto crítico é a escolha entre biblioteca embutida e runtime separado. A integração in-process via Cython pode parecer mais simples no curto prazo, mas aproxima o caminho privilegiado do ambiente do interpretador, o que piora a superfície de erro e complica a disciplina de `fork/execve()`. Um runtime C++ separado, pequeno e especializado, tende a ser a opção mais segura e também a mais alinhada ao modelo mental de um container runtime. O custo adicional de IPC ou invocação controlada costuma ser irrelevante frente ao ganho de isolamento arquitetural.

Por fim, existe uma decisão filosófica de produto. Seu sistema pode optar por ser “flexível por padrão” e deixar endurecimento por conta do usuário, ou pode optar por ser “seguro por padrão” e exigir justificativa para qualquer relaxamento. A pesquisa aqui recomenda fortemente a segunda postura. Em um runtime de containers, permissividade padrão não é neutralidade; é delegação implícita de risco. A combinação `no_new_privs=true`, `capabilities` mínimas, `bounding set` restritivo e `seccomp baseline` é a forma mais coerente de materializar o princípio de mínima autoridade no nível do processo.

## 12. Conclusão

A feature estudada aqui deve ser entendida como o núcleo de segurança de processo de um runtime estilo Docker. Ela não resolve toda a segurança de containers, mas define o momento em que o processo deixa de ser instrumento privilegiado do runtime e passa a ser workload do usuário sob um envelope estrito. Se esse momento estiver bem desenhado, o sistema ganha uma base sólida para evoluir. Se estiver mal desenhado, cada camada adicional de sandbox apenas esconderá um problema fundamental na origem.

No contexto da sua arquitetura, a tradução prática é direta. Python deve ser o lugar da política, do metamodelo, da API e da auditabilidade. Cython deve ser o lugar do contrato tipado e da contenção semântica entre linguagens. C++ deve ser o lugar da sequência irreversível que conversa com kernel, reduz `capabilities`, derruba `bounding set`, aciona `no_new_privs`, instala `seccomp` e faz `execve()`. Essa divisão não é apenas elegante; ela replica uma lição operacional já consolidada no ecossistema Docker/OCI [R17][R19].

A recomendação final é que o projeto implemente essa feature como um protocolo monotônico, imutável e observável, centrado em um `LaunchSecurityPlan` compilado no control plane e consumido por um runtime nativo pequeno e especializado. Comece com um baseline restrito, exija `no_new_privs`, mantenha `ambient=[]`, trate `bounding set` como cerca irreversível do futuro, use `libcap` e `libseccomp` como camadas de segurança operacional sobre as syscalls cruas, e faça o kernel provar o resultado por meio de `/proc/self/status`. Esse caminho não apenas entrega a feature pedida; ele estabelece a cultura correta para todas as próximas features de segurança do seu sistema [R1][R3][R9][R11].

# Referências

- [R1]** Linux kernel documentation — No New Privileges Flag.  
[https://www.kernel.org/doc/html/latest/userspace-api/no\\_new\\_privs.html](https://www.kernel.org/doc/html/latest/userspace-api/no_new_privs.html). Acesso em 12 mar. 2026.
- [R2]** man7 — PR\_SET\_NO\_NEW\_PRIVS(2).  
[https://man7.org/linux/man-pages/man2/PR\\_SET\\_NO\\_NEW\\_PRIVS.2const.html](https://man7.org/linux/man-pages/man2/PR_SET_NO_NEW_PRIVS.2const.html). Acesso em 12 mar. 2026.
- [R3]** man7 — seccomp(2). <https://man7.org/linux/man-pages/man2/seccomp.2.html>. Acesso em 12 mar. 2026.
- [R4]** Linux kernel documentation — Seccomp BPF / seccomp\_filter.  
[https://www.kernel.org/doc/html/latest/userspace-api/seccomp\\_filter.html](https://www.kernel.org/doc/html/latest/userspace-api/seccomp_filter.html). Acesso em 12 mar. 2026.
- [R5]** libseccomp project. <https://github.com/seccomp/libseccomp>. Acesso em 12 mar. 2026.
- [R6]** man7 — seccomp\_init(3). [https://man7.org/linux/man-pages/man3/seccomp\\_init.3.html](https://man7.org/linux/man-pages/man3/seccomp_init.3.html). Acesso em 12 mar. 2026.
- [R7]** man7 — seccomp\_load(3). [https://man7.org/linux/man-pages/man3/seccomp\\_load.3.html](https://man7.org/linux/man-pages/man3/seccomp_load.3.html). Acesso em 12 mar. 2026.
- [R8]** man7 — seccomp\_rule\_add(3).  
[https://man7.org/linux/man-pages/man3/seccomp\\_rule\\_add.3.html](https://man7.org/linux/man-pages/man3/seccomp_rule_add.3.html). Acesso em 12 mar. 2026.
- [R9]** man7 — capabilities(7). <https://man7.org/linux/man-pages/man7/capabilities.7.html>. Acesso em 12 mar. 2026.
- [R10]** man7 — libcap(3). <https://man7.org/linux/man-pages/man3/libcap.3.html>. Acesso em 12 mar. 2026.
- [R11]** man7 — proc\_pid\_status(5).  
[https://man7.org/linux/man-pages/man5/proc\\_pid\\_status.5.html](https://man7.org/linux/man-pages/man5/proc_pid_status.5.html). Acesso em 12 mar. 2026.
- [R12]** Docker Engine security. <https://docs.docker.com/engine/security/>. Acesso em 12 mar. 2026.
- [R13]** Docker run reference — runtime privilege and Linux capabilities.  
<https://docs.docker.com/reference/cli/docker/container/run/#runtime-privilege-and-linux-capabilities>. Acesso em 12 mar. 2026.
- [R14]** Docker run reference — security options / no-new-privileges.  
<https://docs.docker.com/reference/cli/docker/container/run/#security-opt>. Acesso em 12 mar. 2026.
- [R15]** Docker Engine — seccomp security profiles for Docker.  
<https://docs.docker.com/engine/security/seccomp/>. Acesso em 12 mar. 2026.
- [R16]** Docker Engine prior releases and historical notes.  
<https://docs.docker.com/engine/release-notes/prior-releases/>. Acesso em 12 mar. 2026.
- [R17]** OCI Runtime Spec — config.md.  
<https://github.com/opencontainers/runtime-spec/blob/main/config.md>. Acesso em 12 mar. 2026.
- [R18]** OCI Runtime Spec — specs-go/config.go. <https://github.com/opencontainers/runtime-spec/blob/main/specs-go/config.go>. Acesso em 12 mar. 2026.
- [R19]** runc — libcontainer/configs/config.go.  
<https://github.com/opencontainers/runc/blob/main/libcontainer/configs/config.go>. Acesso em 12 mar. 2026.
- [R20]** libcap-ng project and man pages. <https://people.redhat.com/sgrubb/libcap-ng/>. Acesso em 12 mar. 2026.